

Sistemas Tolerantes a Falhas II

Replicação e Redundância

Segunda de três aulas sobre tolerância a falhas em sistemas distribuídos

O que vamos ver hoje

01

Replicação: mascarar, não só detectar

Por que a replicação é a estratégia central de tolerância a falhas

02

Redundância

De hardware, de dados e de tempo — três formas de sobrar recurso para tolerar falha

03

Grupos de processos

Comunicação em grupo confiável: entregar a mesma mensagem a todos os membros vivos

04

Réplicas ativas x passivas

Máquina de estados replicada versus o modelo primary-backup

05

Quorum systems

Votação por maioria e o número de réplicas necessário para tolerar f falhas

Por que replicação é a estratégia central

Detectar uma falha (Aula 21) diz que algo deu errado — mas não resolve, sozinho, a entrega do serviço. A replicação vai além: mantém cópias redundantes de dados e de processos para que a falha de uma parte não interrompa o todo. O objetivo não é só perceber a falha, é mascará-la.

1

Detecção não basta

Saber que um processo falhou não devolve, por si só, a resposta que o cliente esperava. É preciso algo que continue o serviço no lugar dele.

2

Réplicas mascaram a falha

Múltiplas cópias de dados ou instâncias de um processo permitem que, quando uma falha, outra assuma — sem que o serviço pare.

3

Transparência de falha

Do ponto de vista do cliente, o sistema replicado deve seguir respondendo corretamente mesmo com parte das réplicas fora do ar.

Três formas de redundância

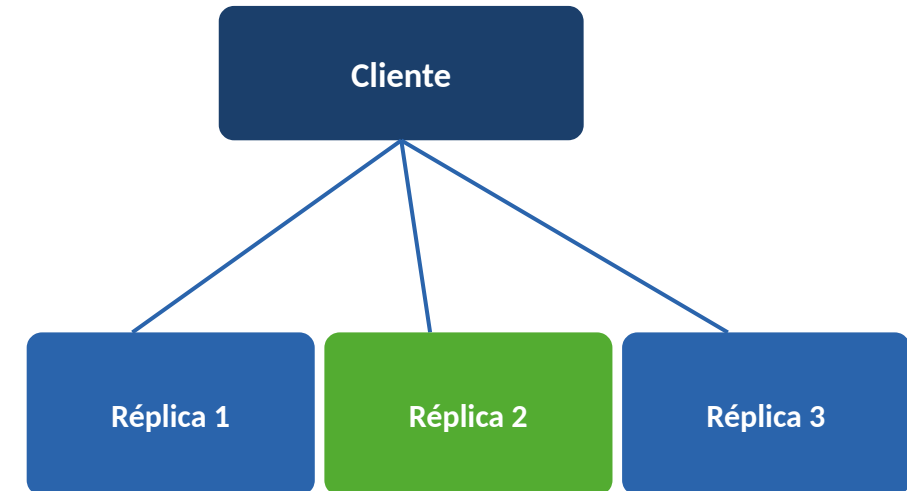
Redundância é sobrar recurso de propósito, para que a perda de uma parte não derrube o sistema. Ela aparece em três dimensões complementares:

Tipo	O que é	Exemplo
Hardware	Componentes físicos duplicados, prontos para assumir se um deles falha.	Discos em RAID, fontes de alimentação redundantes, servidores extras em standby.
Dados (réplicas de estado)	Múltiplas cópias do mesmo dado ou estado mantidas em processos diferentes.	Um banco de dados replicado em três nós; a perda de um nó não perde a informação.
Tempo	Repetir uma ação até que ela tenha sucesso, em vez de manter cópias extras.	Retransmitir uma mensagem perdida; reexecutar uma operação após timeout.

Grupos de processos e comunicação confiável

Um grupo de processos é o conjunto de réplicas que colabora para oferecer o mesmo serviço. Para que a replicação funcione, o grupo precisa de comunicação em grupo confiável: toda mensagem enviada ao grupo deve ser entregue a todos os membros vivos, ou a nenhum — mesmo que o remetente falhe no meio do envio.

- **Grupo fechado** — Só membros do grupo podem enviar mensagens entre si; típico de réplicas cooperando internamente.
- **Grupo aberto** — Processos de fora do grupo (ex.: clientes) também podem enviar mensagens para o grupo.
- **Multicast atômico** — Garante entrega idêntica e na mesma ordem a todos os membros vivos, condição-chave para réplicas convergirem ao mesmo estado.



O mesmo multicast atômico precisa chegar, na mesma ordem, às réplicas 1, 2 e 3 — inclusive se uma delas falhar durante a entrega.

Réplicas ativas x réplicas passivas



Réplicas ativas

Todas as réplicas processam a mesma requisição, na mesma ordem — a chamada máquina de estados replicada. Comunicação em grupo totalmente ordenada garante que todas cheguem ao mesmo estado, de forma simétrica, sem um líder único.

Se uma réplica falha, as demais já têm o estado atualizado: não há failover a esperar.



Réplicas passivas (primary-backup)

Um primário processa cada requisição sozinho e propaga o novo estado aos backups. Mais simples de coordenar: não precisa de acordo entre todas as réplicas a cada requisição.

Se o primário falha, um backup assume (failover) — mas isso depende de detectar a falha com confiança (Aula 21) e introduz um atraso até a retomada do serviço.

Quorum systems: votação por maioria

Nem toda leitura ou escrita precisa envolver todas as réplicas: basta um quorum — um subconjunto que garanta interseção com qualquer outro quorum usado. Isso preserva consistência mesmo tolerando réplicas fora do ar.

Quorum de leitura / escrita

NR e NW: número mínimo de réplicas que precisam participar de uma leitura ou de uma escrita, respectivamente.

Regra de interseção

$NR + NW > N$ garante que toda leitura inclua ao menos uma réplica com a escrita mais recente.

Maioria (majority quorum)

$NW > N/2$ evita que duas escritas concorrentes sejam aceitas sem se sobreporem — a forma mais usada na prática.

Na prática: uma leitura ou escrita só precisa da resposta de um quorum de réplicas — não de todas. O sistema segue disponível mesmo com uma minoria de réplicas falhas ou inacessíveis, sem abrir mão de enxergar a versão mais recente do dado.

Quantas réplicas para tolerar f falhas

O número mínimo de réplicas necessário depende do tipo de falha que o sistema precisa tolerar.

$$2f + 1$$

Falhas crash

Réplicas que simplesmente param de responder. Basta que a maioria continue viva e concordando para o sistema seguir correto — tolera f réplicas paradas com $2f+1$ no total.

$$3f + 1$$

Falhas bizantinas

Réplicas que podem responder qualquer coisa, inclusive de forma maliciosa. É preciso mais réplicas corretas do que o total de réplicas defeituosas conseguiria confundir — daí $3f+1$ no total (sem entrar na prova formal aqui).

O que levar desta aula

1

Replicação mascara falhas em vez de apenas sinalizá-las: o cliente continua sendo atendido mesmo com réplicas fora do ar.

2

Redundância aparece em três dimensões — hardware, dados e tempo — cada uma tolerando um tipo diferente de falha.

3

Réplicas ativas (todas processam) e passivas (primário + backups) são as duas formas de organizar um grupo replicado.

4

Quorum systems e o número de réplicas ($2f+1$ para crash, $3f+1$ para bizantina) equilibram disponibilidade e consistência.

PRÓXIMA AULA

Aula 23 — Sistemas Tolerantes a Falhas III

Recuperação e Estudo de Caso

Depois de ver como mascarar falhas com réplicas ativas, passivas e quorum, falta tratar o que acontece quando uma réplica volta a funcionar: como ela recupera o estado que perdeu e se reintegra ao grupo — fechado com um estudo de caso.

A 3ª e última aula da sequência sobre tolerância a falhas.

Bibliografia desta aula

TANENBAUM, Andrew S.; VAN STEEN, Maarten.

Distributed Systems: Principles and Paradigms (ou edição atual Distributed Systems). Pearson.

Capítulo sobre tolerância a falhas: réplicas ativas e passivas, grupos de processos, quorum systems.

COULOURIS, George et al.

Sistemas Distribuídos: Conceitos e Projeto. 5ª ed. Porto Alegre: Bookman.

Capítulo sobre replicação: comunicação em grupo confiável e consistência de réplicas.

LAMPORT, Leslie; SHOSTAK, Robert; PEASE, Marshall.

"The Byzantine Generals Problem". ACM Transactions on Programming Languages and Systems, 1982.

Artigo original que formaliza o problema das falhas bizantinas e o número de réplicas necessário.